ShadowStack

Predictive Cloud Cost Optimizer

Software Requirements Specification (SRS)

Document Version:	1.0
Status:	Final Draft
Date:	February 24, 2026
Course:	BCA 6th Semester — 3 Credits Project
Methodology:	Agile Scrum — 4 Sprints
Standard:	Based on IEEE 830-1998 SRS Template

Name	Student ID	Role
Aniruddha Bhide	CU23BSC001A	Project Lead & Backend Architect
Sanay Krishna	CU23BCA0049A	Frontend Developer & Visualization Engineer
Arpita M	CU23BCA0011A	Documentation & Quality Support Engineer

This document is prepared as a formal academic deliverable for the BCA 6th Semester capstone project. All technical details are subject to revision during implementation sprints.

Contents

Revision History **3**

1 Introduction **4**
1.1 Purpose . . . 4
1.2 Scope . . . 4
1.3 Definitions, Acronyms and Abbreviations . . . 5
1.4 References . . . 6
1.5 Document Overview . . . 6

2 Overall Description **7**
2.1 Product Perspective . . . 7
2.2 Product Functions — Summary . . . 7
2.3 User Classes and Characteristics . . . 8
2.4 Operating Environment . . . 8
2.5 Design and Implementation Constraints . . . 9
2.6 Assumptions and Dependencies . . . 9

3 Functional Requirements **10**
3.1 FR-WH — Webhook & Event Ingestion . . . 10
3.2 FR-CA — Code Analysis Layer . . . 11
3.3 FR-ML — Machine Learning Pipeline . . . 11
3.4 FR-CE — Cost Estimation Engine . . . 13
3.5 FR-API — RESTful API Layer . . . 15
3.6 FR-UI — Frontend Dashboard . . . 17
3.7 FR-GH — GitHub PR Integration . . . 19
3.8 FR-DB — Data Storage . . . 19

4 Non-Functional Requirements **21**
4.1 Performance Requirements . . . 21
4.2 Reliability and Availability . . . 21
4.3 Security Requirements . . . 21
4.4 Maintainability and Code Quality . . . 22
4.5 Usability Requirements . . . 22
4.6 Portability and Deployment Requirements . . . 23

5 System Interfaces **24**
5.1 User Interface . . . 24
5.2 Hardware Interface . . . 24
5.3 Software Interface . . . 25
5.4 Communication Interface . . . 25

6 Use Case Descriptions **26**
6.1 Actor Summary . . . 26
6.2 UC-01: Submit Pull Request and Receive Cost Prediction . . . 26
6.3 UC-02: View Cost Dashboard . . . 27
6.4 UC-03: Monitor ML Model Performance . . . 28

7 Data Requirements **29**
7.1 Database Schema . . . 29
7.1.1 Table: `usage_data` . . . 29
7.1.2 Table: `predictions` . . . 30

7.1.3 Table: `model_metrics` 31
7.2 Data Volume Estimates 31

8 System Constraints and Assumptions 32
8.1 System Constraints 32
8.2 Assumptions 32

9 Measurable Success Criteria and Definition of Done 33
9.1 Measurable Success Criteria 33
9.2 Definition of Done 33
9.3 Live Demo Scenarios 34

A Technology Stack Summary 35

B Risk Register 35

C Glossary of API Endpoints 36

Revision History

Version	Date	Author	Description
0.1	Feb 7, 2026	Arpita M	Initial draft — structure and scope
0.2	Feb 14, 2026	Aniruddha Bhide	Technical architecture and FR sections added
0.3	Feb 20, 2026	Arpita M	NFR, use-cases, constraints added
1.0	Feb 24, 2026	All Members	Final review, formatting, submission

1 Introduction

1.1 Purpose

This Software Requirements Specification (SRS) document formally describes the functional and non-functional requirements for **ShadowStack — Predictive Cloud Cost Optimizer**. It serves as the primary reference for the development team during all four sprints of the project and provides a baseline for testing, validation, and academic evaluation.

This document is intended for the following audience:

- The development team (Aniruddha Bhide, Sanay Krishna, Arpita M) as the primary implementation reference.
- Academic evaluators and project supervisors assessing project scope and completeness.
- Future developers or contributors extending the system beyond the MVP scope.

1.2 Scope

ShadowStack is an intelligent infrastructure cost prediction system that analyses Python code submitted via GitHub pull requests and produces cost estimates *before* deployment. The system combines static code analysis using Python Abstract Syntax Trees (AST), a machine learning-based cost prediction engine, and a mocked cloud pricing simulation engine to generate actionable cost estimates.

The system is composed of the following high-level components:

1. A **GitHub webhook listener** (FastAPI) that receives pull request events.
2. A **static code analysis** layer that extracts complexity metrics via Python AST.
3. A **machine learning pipeline** (Linear Regression, ARIMA, Random Forest) trained on synthetic and historical usage data.
4. A **cost simulation engine** that maps code complexity to estimated monthly costs using mocked cloud pricing tiers.
5. A **RESTful API layer** exposing prediction, history, and model metrics endpoints.
6. A **React-based interactive dashboard** visualising cost predictions, trends, and model performance.
7. An **automated GitHub PR comment** system that posts cost prediction results to the relevant pull request.

Out of scope for the MVP: Live AWS/GCP/Azure pricing API integration, deep learning models (LSTM/XGBoost), Kubernetes operator, Slack/email alerting, multi-language code analysis, and cloud deployment.

1.3 Definitions, Acronyms and Abbreviations

Term / Acronym	Definition
SRS	Software Requirements Specification
AST	Abstract Syntax Tree — a tree representation of the syntactic structure of source code
API	Application Programming Interface
REST	Representational State Transfer — an architectural style for distributed hypermedia systems
ML	Machine Learning
MAE	Mean Absolute Error — primary model evaluation metric
RMSE	Root Mean Square Error — secondary model evaluation metric
R^2	Coefficient of Determination — goodness-of-fit measure for regression models
PR	Pull Request — a GitHub mechanism for proposing and reviewing code changes
HMAC	Hash-based Message Authentication Code — used to validate GitHub webhook payloads
FastAPI	A modern, high-performance Python web framework for building APIs
Docker	Platform for containerising and running applications in isolated environments
PostgreSQL	Open-source relational database management system
ARIMA	AutoRegressive Integrated Moving Average — a time-series forecasting model
MVP	Minimum Viable Product — the minimum set of features required for a functional deliverable
CI	Continuous Integration — automated build and test pipeline
DoD	Definition of Done — criteria confirming a backlog item is complete
SLO	Service Level Objective — a measurable performance target
USD	United States Dollar — unit for all cost estimates in this system
P95	95th Percentile — a statistical latency measurement threshold

1.4 References

1. ShadowStack Project Proposal — BCA 6th Semester, February 2026.
2. ShadowStack Product Backlog v1.1 — February 2026.
3. ShadowStack Sprint 1 Document — February 2026.
4. IEEE Std 830-1998, *IEEE Recommended Practice for Software Requirements Specifications.*
5. FastAPI Documentation — `https://fastapi.tiangolo.com`
6. React Documentation — `https://react.dev`
7. scikit-learn Documentation — `https://scikit-learn.org`
8. Docker Documentation — `https://docs.docker.com`
9. GitHub Webhooks Documentation — `https://docs.github.com/webhooks`

1.5 Document Overview

This SRS is structured as follows: Section 2 provides an overall product description including system context, assumptions, and constraints. Section 3 details all functional requirements organised by system component. Section 4 covers non-functional requirements. Section 5 describes the system interfaces. Section 6 documents use-case descriptions. Section 7 presents the data requirements. Section 8 covers system constraints and assumptions. Appendices provide supplementary reference material.

2 Overall Description

2.1 Product Perspective

ShadowStack is a novel, standalone predictive system that fills a critical gap in the existing cloud cost management toolchain. Current industry tools such as AWS Cost Explorer, Google Cloud Cost Management, and Kubecost are *reactive* — they report what was spent after resources have been consumed. ShadowStack is *proactive* — it analyses code changes at review time and estimates what those changes *will cost* before they are deployed.

> **Core Differentiator:** ShadowStack embeds cost awareness into the developer workflow at the pull request stage — not the billing stage. The cost prediction comment appears in the GitHub PR before merge, giving developers and engineering managers the information needed to optimise code before money is spent.

The system is designed to run fully on a developer's local machine using Docker Compose, with zero external API dependencies for the MVP. It integrates with GitHub via webhooks and produces output both on the GitHub PR and on an interactive web dashboard.

2.2 Product Functions — Summary

The following table summarises the high-level functions of ShadowStack:

#	Function	Description
F1	Webhook Ingestion	Receive and validate GitHub PR events via HMAC-authenticated POST endpoint.
F2	Code Analysis	Parse submitted Python files using AST to extract structural metrics and compute a cyclomatic complexity score.
F3	Feature Extraction	Transform code metrics and usage data into normalised feature vectors for ML model inference.
F4	Cost Prediction	Run trained ML models (Linear Regression, ARIMA, Random Forest) to predict monthly infrastructure costs in USD.
F5	Cost Simulation	Apply mocked cloud pricing tiers to estimate itemised costs per service (compute, storage, network).
F6	PR Comment Automation	Post a formatted markdown cost-prediction comment to the originating GitHub pull request via the GitHub API.
F7	Dashboard Visualisation	Display cost trends, service breakdowns, predicted vs. actual comparisons, and model metrics on an interactive React dashboard.

#	Function	Description
F8	Historical Data Storage	Persist all analysis results, predictions, usage data, and model metrics in PostgreSQL for future retrieval and trend analysis.
F9	Model Management	Train, evaluate, serialise, version, and load ML models; expose model performance metrics via API.
F10	API Layer	Expose all prediction, history, and metrics data via a documented REST API with OpenAPI specification.

2.3 User Classes and Characteristics

User Class	Characteristics	Primary Interactions
Developer	Submits Python code via GitHub PRs; moderate cloud cost awareness; expects immediate, actionable feedback.	Receives PR cost-prediction comments; uses dashboard to review cost impact of code changes.
Engineering Manager	Budget-conscious; non-technical in infrastructure detail; high-level visibility needed.	Views team cost dashboard; reviews aggregate predicted costs; monitors budget trends.
System Administrator	Technically proficient; responsible for deploying, monitoring, and maintaining ShadowStack.	Accesses model metrics panel; manages Docker deployment; monitors system uptime and API performance.

2.4 Operating Environment

ShadowStack is designed to operate in the following environment:

- **Platform:** Any operating system capable of running Docker (Linux, macOS, Windows with WSL2).
- **Runtime:** Python 3.11+, Node.js 18+, PostgreSQL 15+, Docker 24+.
- **Deployment:** Fully local via Docker Compose; no cloud infrastructure required for MVP.

- **Browser Support:** Chrome (latest), Firefox (latest), Safari (latest) for the React dashboard.
- **GitHub:** Repository access with webhook configuration permissions required for the PR integration feature.
- **Network:** Outbound internet access required only for GitHub API calls (PR comment posting); all other components operate locally.

2.5 Design and Implementation Constraints

1. The system **SHALL** use Python and FastAPI for all backend services — no alternative backend frameworks are permitted in the MVP.
2. All services **SHALL** be containerised and orchestrated using Docker Compose.
3. ML models **SHALL** be trained on synthetic or locally stored historical data — no live cloud API access for pricing data in the MVP.
4. The GitHub PR integration **SHALL** use HMAC-SHA256 signature validation for all incoming webhook payloads.
5. The system **SHALL** be fully deployable from a fresh `git clone` using a single `docker compose up` command.
6. The frontend **SHALL** be built using React 18 — no alternative frontend frameworks are permitted.
7. All API endpoints **SHALL** be documented in OpenAPI format accessible at `/docs`.

2.6 Assumptions and Dependencies

1. The submitted code files are assumed to be valid Python (syntax errors are handled gracefully but not corrected).
2. ML model accuracy depends on the quality and diversity of the synthetic dataset; MAE targets assume at least 500 training samples.
3. The GitHub webhook requires a publicly accessible endpoint; for local development, a tunnelling tool (e.g., ngrok) is assumed to be available.
4. Cloud pricing tiers in the simulation engine are mocked and do not reflect live AWS/GCP/Azure prices.
5. The team assumes a stable internet connection for GitHub API calls (PR comment posting).
6. All development and testing occurs on local machines; no cloud deployment infrastructure is available or required.

3 Functional Requirements

Functional requirements are identified using the format **FR-XX-YY** where **XX** denotes the component group and **YY** is the sequential number within that group. Priority levels: **HIGH** = MVP-critical, **MED** = MVP but not blocking, **LOW** = desirable.

3.1 FR-WH — Webhook & Event Ingestion

ID	Requirement	Acceptance Criteria	Priority
FR-WH-01	The system **SHALL** expose a `POST /webhook/github` endpoint that accepts GitHub pull request event payloads in JSON format.	Endpoint returns HTTP 200 for valid PR events.	HIGH
FR-WH-02	The system **SHALL** validate the HMAC-SHA256 signature on all incoming webhook payloads using the `X-Hub-Signature-256` header.	Invalid signatures are rejected with HTTP 401.	HIGH
FR-WH-03	The system **SHALL** trigger the code analysis pipeline automatically on receipt of a valid `pull_request` event with action `opened` or `synchronize`.	Pipeline is initiated within 2 seconds of receiving a valid event.	HIGH
FR-WH-04	The system **SHALL** log all received webhook payloads to the PostgreSQL database, including timestamp, event type, repository name, and PR number.	DB entry confirmed for each valid event.	HIGH
FR-WH-05	The system **SHALL** respond to any webhook event within 30 seconds, returning an appropriate HTTP status code.	All valid payloads acknowledged within 30 s under normal load.	HIGH
FR-WH-06	The system **SHOULD** handle malformed or unsupported event types gracefully, returning HTTP 400 without crashing.	No unhandled exceptions on malformed input.	MED

3.2 FR-CA — Code Analysis Layer

ID	Requirement	Acceptance Criteria	Priority
FR-CA-01	The system **SHALL** parse Python source code files using the Python `ast` module to extract structural information.	Parser returns node counts, function list, class list, and import list for any valid Python file.	HIGH
FR-CA-02	The system **SHALL** extract the following metrics from each analysed file: number of functions, number of classes, number of loops (`for`/`while`), number of conditional branches (`if`/`elif`/`else`), number of imports, and total AST node count.	All six metric fields populated for valid Python input.	HIGH
FR-CA-03	The system **SHALL** compute a cyclomatic complexity score (scale 1–10) for each analysed file based on branch and loop counts.	Score returned for all valid files; formula documented; score correlates with branch and loop count.	HIGH
FR-CA-04	The system **SHALL** handle Python syntax errors gracefully, returning an error status and descriptive message without crashing the analysis pipeline.	SyntaxError files return error response; subsequent files processed normally.	HIGH
FR-CA-05	The system **SHOULD** support batch analysis of multiple Python files submitted in a single PR.	Multiple files analysed; per-file and aggregate metrics returned.	MED

3.3 FR-ML — Machine Learning Pipeline

ID	Requirement	Acceptance Criteria	Priority
FR-ML-01	The system **SHALL** include a synthetic dataset generation script that produces a minimum of 500 labelled training samples.	Dataset contains columns: complexity score, resource patterns, monthly cost (USD); reproducible with a fixed seed; stored in PostgreSQL and CSV.	HIGH
FR-ML-02	The system **SHALL** implement a feature extraction pipeline that accepts code complexity metrics and usage data and outputs normalised feature vectors.	Pipeline output is serialisable with `joblib`; unit tested for correctness; shared between training and inference.	HIGH
FR-ML-03	The system **SHALL** train and evaluate a Linear Regression model as the baseline cost predictor.	Trained on 80/20 train-test split; MAE and RMSE computed and stored in DB; model serialised to `.pkl`.	HIGH
FR-ML-04	The system **SHALL** train and evaluate an ARIMA time-series model for cost trend forecasting.	Model fitted on historical usage sequences; 30-day forecast horizon; MAE <15% on held-out test set; serialised.	HIGH

ID	Requirement	Acceptance Criteria	Priority
FR-ML-05	The system **SHALL** train and evaluate a Random Forest ensemble model for cost prediction.	Trained with cross-validation; feature importances logged; MAE and RMSE computed; serialised.	HIGH
FR-ML-06	The system **SHALL** implement a unified model evaluation framework that computes MAE, RMSE, and R^2 for any serialised model.	Evaluation script runs on any `.pkl` model; results stored in `model_metrics` table; best model flagged automatically.	HIGH
FR-ML-07	The best-performing model **SHALL** achieve a Mean Absolute Error (MAE) of less than 15% on the held-out test dataset.	Verified by the evaluation framework post-training.	HIGH
FR-ML-08	The system **SHALL** implement model versioning: each saved model **SHALL** be stored with a version identifier and metadata in the database.	Models saved as `model_v{n}_{type}.pkl`; version metadata in DB; API loads latest model at startup.	MED
FR-ML-09	The system **SHALL** include data preprocessing utilities covering: normalisation, missing-value imputation, and feature scaling.	Normaliser, imputer, and scaler implemented; edge cases handled; unit tested.	HIGH

3.4 FR-CE — Cost Estimation Engine

ID	Requirement	Acceptance Criteria	Priority
FR-CE-01	The system **SHALL** implement a cost simulation engine with configurable, mocked cloud pricing tiers for compute, storage, and network resources.	Pricing configuration loaded from YAML; engine returns itemised cost breakdown per service; unit tested.	HIGH
FR-CE-02	The system **SHALL** map code complexity scores to estimated monthly infrastructure costs in USD using documented mapping rules.	Function returns monthly USD cost given a complexity score and resource usage pattern; mapping rules documented; output range validated in tests.	HIGH
FR-CE-03	The system **SHALL** produce an itemised cost breakdown showing estimated costs for at least three service categories: compute, storage, and network.	Each prediction response includes per-category cost figures summing to the total predicted cost.	HIGH
FR-CE-04	The cost estimation result **SHALL** include a delta value comparing the predicted cost of the current code change against the established baseline cost.	Delta value (positive or negative USD/month) included in all prediction responses.	HIGH

ID	Requirement	Acceptance Criteria	Priority
FR-CE-05	The system **SHOULD** generate at least one optimisation recommendation alongside each cost estimate.	Recommendation text present in prediction response when cost exceeds a configurable threshold.	MED

3.5 FR-API — RESTful API Layer

ID	Requirement	Acceptance Criteria	Priority
FR-API-01	The system **SHALL** expose a health check endpoint `GET /health` that returns HTTP 200 and service status.	Returns `{"status": "ok"}` and HTTP 200 when all services are running.	HIGH
FR-API-02	The system **SHALL** expose a prediction endpoint `POST /api/predict` that accepts a code payload and returns a cost prediction.	Returns JSON with predicted monthly cost, itemised breakdown, delta, and model version; P95 response time <500 ms.	HIGH

ID	Requirement	Acceptance Criteria	Priority
FR-API-03	The system **SHALL** expose a cost history endpoint `GET /api/costs/history` that returns paginated historical cost records.	Supports date range and service filters; pagination parameters (`page`, `limit`); response <500 ms for 1,000 records.	HIGH
FR-API-04	The system **SHALL** expose a model metrics endpoint `GET /api/model/metrics` that returns MAE, RMSE, and R^2 for all trained models.	Returns metrics for all available models with version identifiers; supports model version selector.	HIGH
FR-API-05	All API endpoints **SHALL** be documented in OpenAPI 3.0 format, accessible via the `/docs` path.	All endpoints, request schemas, response schemas, and error codes documented at `/docs`.	HIGH
FR-API-06	All API endpoints **SHALL** return appropriate HTTP status codes: 200 (success), 400 (bad request), 401 (unauthorised), 404 (not found), 422 (validation error), 500 (server error).	Each error scenario returns the correct status code and a descriptive error message.	HIGH
FR-API-07	The system **SHOULD** export a Postman collection covering all API endpoints with example requests and responses.	Postman collection JSON committed to the repository.	MED

3.6 FR-UI — Frontend Dashboard

ID	Requirement	Acceptance Criteria	Priority
FR-UI-01	The system **SHALL** provide an interactive cost trend line chart displaying historical and predicted costs over time.	Chart.js/D3.js line chart; x-axis: date; y-axis: USD; predicted vs. actual series; tooltips; time-range selector (7d/30d/90d).	HIGH
FR-UI-02	The system **SHALL** provide a service-wise cost breakdown donut chart.	Donut chart with a minimum of 3 segments; legend with percentages; tooltips; data sourced from `/api/costs/history`.	HIGH
FR-UI-03	The system **SHALL** provide a predicted vs. actual cost comparison grouped bar chart.	Grouped bars per time bucket; variance percentage displayed; colour-coded above/below threshold.	HIGH
FR-UI-04	The system **SHALL** provide an ML model performance metrics panel.	Displays MAE, RMSE, and R^2 for all models as metric cards with colour thresholds; model version selector present.	HIGH

ID	Requirement	Acceptance Criteria	Priority
FR-UI-05	The system **SHALL** provide date-range filter controls applicable to all charts.	Date picker present on all chart views; selecting a range re-fetches data for all charts; default range: last 30 days; filter state reflected in URL parameters.	MED
FR-UI-06	The system **SHALL** provide a persistent header and sidebar with working navigation links to all views.	Header with logo; sidebar links to all views; active route highlighted; mobile hamburger menu; renders without errors.	HIGH
FR-UI-07	The dashboard **SHALL** refresh automatically to show new PR analysis results without a manual page reload.	Polling interval 30 seconds or WebSocket connection; new data reflected in charts; loading spinner displayed during refresh.	MED
FR-UI-08	The system provide interactive data filtering and sorting on the cost records table.	Service drop-down filter; sortable cost table; combined filter and sort functions correctly.	MED

ID	Requirement	Acceptance Criteria	Priority
FR-UI-09	The system **SHOULD** provide a team cost dashboard view with aggregate predicted costs filterable by date.	Dedicated view with aggregate totals; filterable by date range; cost totals card on main dashboard.	MED

3.7 FR-GH — GitHub PR Integration

ID	Requirement	Acceptance Criteria	Priority
FR-GH-01	The system **SHALL** automatically post a cost-prediction comment to the originating GitHub pull request upon completion of the analysis pipeline.	Comment posted to the correct PR within 30 seconds of PR open or update event.	HIGH
FR-GH-02	The PR comment **SHALL** be formatted as a markdown table containing: current baseline cost, predicted monthly cost, cost delta, and at least one optimisation recommendation.	Comment renders correctly as a formatted markdown table on GitHub.	HIGH
FR-GH-03	The system **SHALL** update the PR comment on subsequent pushes to the same PR (synchronize event), replacing the previous comment.	Only one ShadowStack comment present per PR; comment updated not duplicated on re-push.	HIGH

3.8 FR-DB — Data Storage

ID	Requirement	Acceptance Criteria	Priority
FR-DB-01	The system **SHALL** persist all prediction results, including input metrics, predicted cost, itemised breakdown, model version, and timestamp, in the `predictions` table.	All fields populated and retrievable via `/api/costs/history`.	HIGH
FR-DB-02	The system **SHALL** persist all usage data records (timestamps, service names, cost values) in the `usage_data` table.	Data quality checks pass; table queryable by date range and service name.	HIGH
FR-DB-03	The system **SHALL** persist all model training results (MAE, RMSE, R^2, model version, training date) in the `model_metrics` table.	All metric fields populated after each training run.	HIGH
FR-DB-04	The system **SHALL** apply all database schema migrations automatically at startup.	Fresh container starts with all tables present and schema up to date.	HIGH

4 Non-Functional Requirements

4.1 Performance Requirements

ID	Requirement	Target	Measurement Method
NFR-P-01	PR analysis end-to-end (webhook receipt to PR comment posted)	<30 seconds	Measured in Sprint 4 integration test
NFR-P-02	API endpoint P95 response time	<500 ms	Load test with 10 concurrent requests
NFR-P-03	Dashboard initial page load time	<2 seconds	Lighthouse or browser devtools
NFR-P-04	`POST /api/predict` response time	<2 seconds	Measured with Postman automated tests
NFR-P-05	Docker Compose full stack startup time	<60 seconds on a clean machine	Timed from `docker compose up` to all containers healthy

4.2 Reliability and Availability

ID	Requirement	Target
NFR-R-01	System uptime during evaluation period	95%
NFR-R-02	The system **SHALL** recover automatically from a PostgreSQL container restart without data loss.	<30 s recovery
NFR-R-03	Failed webhook events **SHALL** not crash the API server; errors logged and appropriate status code returned.	Zero unhandled exceptions

4.3 Security Requirements

ID	Requirement
NFR-S-01	All sensitive configuration values (DB credentials, GitHub tokens, webhook secrets) **SHALL** be stored in environment variables, never hard-coded.
NFR-S-02	A `.env.example` file **SHALL** be committed to the repository with placeholder values; the real `.env` file **SHALL** be listed in `.gitignore`.
NFR-S-03	The webhook endpoint **SHALL** validate HMAC-SHA256 signatures on every incoming request and reject invalid requests with HTTP 401.
NFR-S-04	The system **SHOULD** not expose raw stack traces in API error responses in production mode.

4.4 Maintainability and Code Quality

ID	Requirement
NFR-M-01	Unit test coverage **SHALL** be 70% across all Sprint modules (backend, ML, and frontend combined).
NFR-M-02	Python code **SHALL** follow PEP 8 naming and formatting conventions; all linting checks **SHALL** pass.
NFR-M-03	JavaScript/React code **SHALL** follow camelCase naming; no unused variables or imports; ESLint checks **SHALL** pass.
NFR-M-04	No commented-out code or unresolved `TODO` comments **SHALL** be present in the final submission.
NFR-M-05	All code changes **SHALL** be merged to `main` via reviewed GitHub pull requests; direct pushes to `main` are not permitted.

4.5 Usability Requirements

ID	Requirement
NFR-U-01	The dashboard **SHALL** be responsive and usable at viewport widths from 320px to 1920px.
NFR-U-02	All dashboard text and interactive elements **SHOULD** meet WCAG 2.1 AA colour contrast requirements.

ID	Requirement
NFR-U-03	All charts **SHALL** include axis labels, tooltips, and a legend; data points **SHALL** be distinguishable without relying solely on colour.
NFR-U-04	Loading states **SHALL** be shown for all asynchronous operations; error states **SHALL** be communicated clearly to the user.
NFR-U-05	The dashboard **SHALL** function correctly on the latest stable versions of Chrome and Firefox.

4.6 Portability and Deployment Requirements

ID	Requirement
NFR-D-01	The complete system **SHALL** be deployable with a single `docker compose up` command from a fresh `git clone` on any Docker-capable machine.
NFR-D-02	A deployment guide **SHALL** document all prerequisite software, environment variable configuration, and verification steps.
NFR-D-03	The system **SHALL** function on Linux, macOS, and Windows (with WSL2 or Docker Desktop).

5 System Interfaces

5.1 User Interface

The user interface is a React 18 single-page application (SPA) served on port 3000. It communicates exclusively with the FastAPI backend via the Axios HTTP client. The UI comprises the following views:

- **Main Dashboard:** Summary cards (total predicted cost, delta, model accuracy) and cost trends line chart.
- **Cost Trends View:** Full interactive line chart with time-range filters (7d / 30d / 90d).
- **Service Breakdown View:** Donut chart of cost per service category with interactive legends.
- **Comparison View:** Grouped bar chart of predicted vs. actual cost per time bucket.
- **Model Metrics View:** Cards displaying MAE, RMSE, and R^2 with model version selector.
- **Team Cost View:** Aggregate predicted costs by team/date range.

5.2 Hardware Interface

No specialised hardware is required. The system runs on any commodity laptop or desktop with at least:

- 4 GB RAM (8 GB recommended for Docker multi-container stack)
- 10 GB free disk space
- Intel/AMD 64-bit processor (Apple Silicon supported via Docker Desktop)

5.3 Software Interface

Interface	Type	Purpose
GitHub Webhooks API	HTTP/JSON (inbound)	Receives PR events for analysis triggering
GitHub REST API v3	HTTP/JSON (outbound)	Posts cost-prediction comments to PRs
PostgreSQL 15	TCP socket	Persistent data storage for all application data
React 18 + Vite	HTTP (port 3000)	Frontend SPA served to browser clients
FastAPI (Uvicorn)	HTTP (port 8000)	Backend API server
Docker Compose	Local networking	Service orchestration and inter-container communication
scikit-learn / statsmodels	Python module	ML model training and inference

5.4 Communication Interface

- **Webhook events:** Received over HTTPS from GitHub servers; validated via HMAC-SHA256.
- **API communication:** All internal API calls use HTTP/1.1 on the Docker internal network.
- **GitHub PR comment posting:** Outbound HTTPS to `api.github.com` using a personal access token stored in environment variables.
- **Real-time dashboard updates:** HTTP polling at 30-second intervals from the React frontend to `/api/costs/history` and `/api/model/metrics`.

6 Use Case Descriptions

6.1 Actor Summary

Actor	Description
Developer	A software engineer who opens or updates GitHub pull requests containing Python code. Primary beneficiary of PR cost-prediction comments.
Engineering Manager	Reviews the team cost dashboard to monitor aggregate predicted infrastructure spend.
System Administrator	Deploys, configures, and monitors the ShadowStack system. Reviews model performance metrics.
GitHub (External)	External actor that sends PR webhook events to the ShadowStack webhook listener and receives PR comment API calls.

6.2 UC-01: Submit Pull Request and Receive Cost Prediction

Use Case ID: UC-01

Title: Submit Pull Request and Receive Cost Prediction

Actors: Developer (primary), GitHub (external), ShadowStack System

Preconditions: ShadowStack is running; GitHub webhook is configured on the repository; developer has opened or pushed to a PR.

Main Flow:

1. Developer opens or updates a GitHub PR containing Python code changes.
2. GitHub sends a `pull_request` webhook event (action: `opened` or `synchronize`) to `POST /webhook/github`.
3. System validates the HMAC-SHA256 signature.
4. System logs the payload to the `usage_data` table.
5. System fetches the changed Python files from the PR.
6. System runs AST-based code analysis and computes complexity score.
7. System runs feature extraction pipeline to generate normalised feature vector.
8. System invokes the best-performing ML model via `POST /api/predict`.
9. System runs the cost simulation engine and generates an itemised cost breakdown.

10. System posts a formatted markdown comment to the PR via the GitHub API.
11. Developer sees the cost prediction comment on their PR within 30 seconds.

Alternative Flow:

- If HMAC validation fails → return HTTP 401; stop processing.
- If Python file has syntax errors → log error; skip file; continue with remaining files.
- If GitHub API call fails → log error; store prediction in DB; retry once.

Postconditions: Prediction stored in DB; PR comment posted; dashboard updated.

Success Criteria: PR comment visible within 30 seconds.

6.3 UC-02: View Cost Dashboard

Use Case ID: UC-02

Title: View Interactive Cost Dashboard

Actors: Developer, Engineering Manager

Preconditions: ShadowStack is running; at least one prediction has been stored.

Main Flow:

1. User navigates to the ShadowStack dashboard in their browser.
2. Dashboard loads and displays the main view with summary cost cards.
3. User selects a time range using the date-range picker.
4. Dashboard re-fetches data from `GET /api/costs/history` and updates all charts.
5. User navigates to the Service Breakdown view; donut chart renders.
6. User navigates to the Comparison view; grouped bar chart renders with predicted vs. actual data.
7. Dashboard auto-refreshes every 30 seconds; new data appears without manual reload.

Postconditions: User has current cost visualisation data displayed.

Success Criteria: Dashboard loads in <2 seconds; all charts render with real data.

6.4 UC-03: Monitor ML Model Performance

Use Case ID: UC-03

Title: Monitor ML Model Performance Metrics

Actors: System Administrator

Preconditions: At least one model has been trained and evaluated.

Main Flow:

1. System Administrator navigates to the Model Metrics view on the dashboard.
2. Dashboard calls `GET /api/model/metrics` and renders metric cards.
3. Administrator reviews MAE, RMSE, and R^2 values for all available model versions.
4. Administrator selects a different model version using the version selector.
5. Dashboard updates to show metrics for the selected model version.

Postconditions: Administrator has reviewed model performance; can identify if retraining is needed.

Success Criteria: Metrics displayed for all models; selector functional.

7 Data Requirements

7.1 Database Schema

The system uses three primary tables in PostgreSQL:

7.1.1 Table: `usage_data`

Column	Type	Nullable	Description
id	BIGSERIAL	NOT NULL	Primary key
timestamp	TIMESTAMPTZ	NOT NULL	Time of usage record
service_name	VARCHAR(100)	NOT NULL	Cloud service name (e.g., compute, storage)
cost_usd	NUMERIC(12,4)	NOT NULL	Actual cost in USD
region	VARCHAR(50)	NULL	Cloud region identifier
resource_type	VARCHAR(100)	NULL	Resource type (e.g., t3.medium, S3)
created_at	TIMESTAMPTZ	NOT NULL	Row insertion timestamp

7.1.2 Table: `predictions`

Column	Type	Nullable	Description
id	BIGSERIAL	NOT NULL	Primary key
pr_number	INTEGER	NULL	GitHub PR number (if triggered by webhook)
repository	VARCHAR(200)	NULL	GitHub repository full name
complexity_score	NUMERIC(5,2)	NOT NULL	Computed cyclomatic complexity (1–10)
predicted_cost_usd	NUMERIC(12,4)	NOT NULL	Predicted monthly cost in USD
baseline_cost_usd	NUMERIC(12,4)	NULL	Baseline cost for delta calculation
cost_delta_usd	NUMERIC(12,4)	NULL	Predicted cost minus baseline
compute_cost_usd	NUMERIC(12,4)	NULL	Itemised compute component
storage_cost_usd	NUMERIC(12,4)	NULL	Itemised storage component
network_cost_usd	NUMERIC(12,4)	NULL	Itemised network component
model_version	VARCHAR(50)	NOT NULL	Model version used for this prediction
recommendation	TEXT	NULL	Optimisation recommendation text
created_at	TIMESTAMPTZ	NOT NULL	Prediction timestamp

7.1.3 Table: `model_metrics`

Column	Type	Nullable	Description
id	BIGSERIAL	NOT NULL	Primary key
model_version	VARCHAR(50)	NOT NULL	Model version identifier
model_type	VARCHAR(50)	NOT NULL	Model type: linear_regression / arima / random_forest
mae	NUMERIC(10,6)	NOT NULL	Mean Absolute Error
rmse	NUMERIC(10,6)	NOT NULL	Root Mean Square Error
r2_score	NUMERIC(10,6)	NOT NULL	R-squared coefficient of determination
is_best	BOOLEAN	NOT NULL	Flag: TRUE if this is the currently deployed best model
trained_at	TIMESTAMPTZ	NOT NULL	Training completion timestamp
training_samples	INTEGER	NULL	Number of samples used in training

7.2 Data Volume Estimates

Table	Expected Volume (MVP)	Notes
usage_data	500 synthetic rows; real records added per PR	Synthetic data generated in Sprint 2
predictions	1 row per PR analysis	Approx. 10–50 records during evaluation
model_metrics	3–10 rows total	One row per training run per model type

8 System Constraints and Assumptions

8.1 System Constraints

1. **Language Constraint:** Only Python source files are supported for static code analysis in the MVP. JavaScript, Go, Java, and other languages are out of scope.
2. **Pricing Data Constraint:** Cloud pricing data is mocked and loaded from a static YAML configuration file. Real-time AWS/GCP/Azure pricing API integration is explicitly excluded from the MVP.
3. **Deployment Constraint:** The system is designed exclusively for local deployment via Docker Compose. Cloud deployment (AWS ECS, GKE, etc.) is a post-MVP enhancement.
4. **Model Constraint:** Only three ML model types are supported in the MVP: Linear Regression, ARIMA, and Random Forest. Deep learning models (LSTM, XGBoost) are post-MVP.
5. **Single Repository Constraint:** The webhook listener supports one configured GitHub repository per deployment instance in the MVP.
6. **Dataset Constraint:** ML model performance is constrained by the synthetic dataset. Accuracy guarantees apply only to the held-out portion of this dataset, not to arbitrary production code.

8.2 Assumptions

1. All Python files submitted via PRs are syntactically valid UTF-8 encoded Python 3 source files. Syntax errors are handled gracefully but the system does not support Python 2 syntax.
2. The team has access to a GitHub account and repository with webhook configuration permissions and the ability to generate a personal access token.
3. For local webhook testing, a tunnelling tool (e.g., ngrok, smee.io) is available to expose the local endpoint to GitHub.
4. The mocked pricing tiers in the YAML configuration are representative enough to demonstrate the cost prediction concept for academic evaluation purposes.
5. The synthetic dataset generator produces sufficiently varied data to allow meaningful ML model training and differentiation between model types.
6. Evaluators will run the system locally via Docker Compose following the provided deployment guide.

9 Measurable Success Criteria and Definition of Done

9.1 Measurable Success Criteria

Metric	Target	Related Requirements
ML Model Accuracy (MAE)	<15% on test set	FR-ML-07, NFR-P-04
PR Analysis End-to-End Time	<30 seconds	FR-WH-05, FR-GH-01, NFR-P-01
Dashboard Page Load Time	<2 seconds	FR-UI, NFR-P-03
API Response Time (P95)	<500 ms	FR-API-02, NFR-P-02
System Uptime (evaluation period)	95%	NFR-R-01
Code Test Coverage	70%	NFR-M-01
Documentation Completeness	100% of all modules	FR-API-05, all doc FRs
MVP Feature Completion	100% of PBI-001 to PBI-060	All FRs
Minimum Training Dataset Size	500 samples	FR-ML-01
Number of Trained Models	3 (LR, ARIMA, RF)	FR-ML-03 to FR-ML-05

9.2 Definition of Done

A feature or backlog item is considered **Done** when all of the following conditions are met:

1. All acceptance criteria for the item are verified and signed off by the item owner.
2. Code is merged to the `main` branch via a reviewed GitHub pull request.
3. Relevant unit and/or integration tests are written and pass in CI.
4. No new linting or type-checking errors are introduced.
5. The feature is demonstrable in the running Docker Compose environment.
6. Impacted documentation is updated (README, API docs, or user guide as applicable).
7. The item is marked as Done on the Agile project board.

9.3 Live Demo Scenarios

The final demonstration must successfully cover all six scenarios:

1. Opening a GitHub pull request with real Python code changes and observing the webhook trigger.
2. The system automatically analysing the code and posting a formatted cost-prediction comment to the PR.
3. The dashboard displaying predicted vs. current costs updated in real time after the PR analysis.
4. The service-wise cost breakdown donut chart rendered with live data.
5. The historical cost trends chart with time-range filtering demonstrating data filtering.
6. The ML model performance metrics panel showing MAE, RMSE, and R^2 for all trained models.

A Technology Stack Summary

Layer	Technologies	Version (Target)
Backend Framework	Python, FastAPI, Uvicorn	Python 3.11+, FastAPI 0.110+
ML / Data Science	scikit-learn, statsmodels, pandas, NumPy	scikit-learn 1.4+, pandas 2.x
Database	PostgreSQL	PostgreSQL 15
ORM / Migrations	SQLAlchemy / Alembic	SQLAlchemy 2.x
Frontend	React 18, Vite, React Router v6	React 18.x
HTTP Client	Axios	Axios 1.x
Visualisation	Chart.js or D3.js	Chart.js 4.x / D3 v7
Containerisation	Docker, Docker Compose	Docker 24+, Compose v2
Testing (Backend)	pytest, pytest-cov	pytest 8.x
Testing (Frontend)	Jest, React Testing Library	Jest 29+, RTL 14+
Version Control	GitHub	—
API Documentation	OpenAPI 3.0 (built-in FastAPI)	—

B Risk Register

#	Risk	Likelihood	Impact	Mitigation
R1	ML model fails to achieve MAE <15%	Medium	High	Increase synthetic dataset to 1,000+ samples; add feature engineering; try additional model configurations.
R2	GitHub webhook not accessible from local machine	Medium	Medium	Use ngrok or smee.io for tunnelling; document setup in deployment guide.
R3	GitHub API rate limiting blocks PR comment posting	Low	Medium	Implement exponential backoff; cache GitHub tokens.
R4	Docker inter-container networking issues	Low	High	Declare explicit container names; test on a clean machine in Sprint 4.

#	Risk	Likelihood	Impact	Mitigation
R5	Sprint 2 ML tasks underestimated (63 SP in 1.5 weeks)	Medium	High	Parallelise data preparation (Arpita) and model development (Aniruddha); reduce evaluation depth if needed.
R6	React chart library integration issues	Low	Medium	Chart library set up in Sprint 1; mock data integration tested before Sprint 3 live data work.

C Glossary of API Endpoints

Method	Endpoint	Description
GET	`/health`	System health check; returns service status.
POST	`/webhook/github`	GitHub PR event ingestion endpoint.
POST	`/api/predict`	Accepts Python code; returns cost prediction.
GET	`/api/costs/history`	Returns paginated historical prediction records.
GET	`/api/model/metrics`	Returns MAE, RMSE, R^2 for all model versions.
GET	`/docs`	Interactive OpenAPI documentation (Swagger UI).

ShadowStack — Software Requirements Specification · Version 1.0 · February 24, 2026
BCA 6th Semester Capstone Project · Aniruddha Bhide · Sanay Krishna · Arpita M
Living document — subject to revision in subsequent sprints.